

**UNAM**

**Universidad Nacional Autónoma de México**

Facultad de Ciencias

**Fundamentos de Bases de Datos**

## **Práctica 01**

Instalación de Docker y del SGBD PostgreSQL

**Bitácora individual**

**Integrante:**

Omar Alejandro Juárez Larrondo

Número de cuenta: 322245244 | Equipo: ÑIPGG

Semestre: 2027-I

## Índice

---

|                                                     |          |
|-----------------------------------------------------|----------|
| <b>1. Datos del entorno y de la instalación</b>     | <b>1</b> |
| <b>2. Instalación de Docker</b>                     | <b>2</b> |
| 2.1. Inspección y adaptación a Fedora . . . . .     | 2        |
| 2.2. Prueba funcional . . . . .                     | 2        |
| <b>3. Instalación y configuración de PostgreSQL</b> | <b>3</b> |
| 3.1. Descarga de la imagen . . . . .                | 3        |
| 3.2. Credencial y creación del contenedor . . . . . | 3        |
| 3.3. Comprobación con psql . . . . .                | 4        |
| <b>4. Conexión con DBeaver</b>                      | <b>5</b> |
| 4.1. Instalación del cliente . . . . .              | 5        |
| <b>5. Problemas encontrados y comentarios</b>       | <b>7</b> |
| 5.1. Incidencias reales . . . . .                   | 7        |
| 5.2. Comentarios finales . . . . .                  | 7        |

## 1 Datos del entorno y de la instalación

La práctica se realizó en una estación de trabajo con Fedora Linux 44, una distribución GNU/Linux de 64 bits. El equipo utilizó el núcleo 7.1.13-200.fc44.x86\_64. Antes de efectuar cambios se inspeccionó el estado del sistema para distinguir entre componentes ausentes y componentes ya instalados. Esta comprobación evitó reemplazar innecesariamente una instalación funcional de Docker.

| Componente              | Versión registrada                            |
|-------------------------|-----------------------------------------------|
| Sistema operativo       | Fedora Linux 44 (Workstation Edition), x86_64 |
| Docker Engine y cliente | 29.7.2                                        |
| containerd              | 2.3.4                                         |
| Docker Buildx           | 0.36.1                                        |
| PostgreSQL              | 18.6, contenido en la imagen postgres:18      |
| DBeaver Community       | 26.2.0                                        |
| Controlador JDBC        | PostgreSQL JDBC 42.7.13                       |

Tabla 1: Resumen del entorno utilizado durante la práctica.

```
PRÁCTICA 01 - DATOS DEL ENTORNO

Sistema operativo: Fedora Linux 44 (Workstation Edition)
Arquitectura:      x86_64
Kernel:           7.1.13-200.fc44.x86_64

Versiones instaladas:
Docker version 29.7.2, build 1.fc44
moby-engine-29.7.2-1.fc44.x86_64
docker-cli-29.7.2-1.fc44.x86_64
containerd-2.3.4-1.fc44.x86_64
docker-buildx-0.36.1-1.fc44.x86_64
dbeaver-ce-26.2.0-stable.x86_64

Servicio Docker:   active
```

Figura 1: Identificación del sistema, paquetes instalados y estado del servicio Docker.

El tiempo transcurrido entre la primera comprobación del entorno y la conexión correcta desde DBeaver fue de aproximadamente cinco minutos. Este valor corresponde al trabajo activo de habilitación, descarga, creación y prueba; no incluye el tiempo posterior destinado a organizar capturas, redactar la bitácora ni revisar el documento final.

**Criterio de seguridad**

La contraseña de PostgreSQL se generó de manera aleatoria y se almacenó temporalmente en un archivo con permisos restrictivos. Por esta razón, la credencial no aparece en comandos, capturas ni fragmentos publicados.

## 2 Instalación de Docker

### 2.1 Inspección y adaptación a Fedora

Las instrucciones de referencia presentan procedimientos para Debian y Ubuntu. En este equipo no resultaba correcto repetirlos, pues Docker ya se encontraba instalado mediante los paquetes propios de Fedora: `moby-engine`, `docker-cli`, `containerd` y `docker-buildx`. Por tanto, el procedimiento se concentró en comprobar los paquetes, habilitar el servicio y validar el funcionamiento del motor.

El servicio se configuró para iniciarse automáticamente y se arrancó en la sesión actual:

```
1 $ sudo systemctl enable --now docker
2 $ systemctl is-enabled docker
3 enabled
4 $ systemctl is-active docker
5 active
```

El usuario ya figuraba como integrante del grupo `docker`; sin embargo, la sesión desde la que se inició la práctica todavía no había incorporado esa pertenencia. Para aplicar el grupo sin reiniciar el equipo se utilizó una subsesión equivalente a volver a iniciar sesión:

```
1 $ getent group docker
2 docker:x:971:omar
3 $ sg docker -c "docker version"
```

**Implicación del grupo docker**

La pertenencia al grupo `docker` permite controlar el demonio y, en la práctica, concede capacidades comparables con privilegios administrativos. El acceso debe asignarse únicamente a usuarios de confianza.

### 2.2 Prueba funcional

La comunicación entre el cliente y el servidor se comprobó con sus versiones. Después se ejecutó la imagen mínima `hello-world`. Docker descargó la imagen, creó un contenedor efímero y devolvió el mensaje de confirmación, lo que demostró el funcionamiento completo de la ruta cliente–demonio–registro.

```
1 $ docker version
2 $ docker run --rm hello-world
```

```
PRÁCTICA 01 - VALIDACIÓN DE DOCKER

$ docker version
Cliente: 29.7.2\nServidor: 29.7.2

$ docker run --rm hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

Figura 2: Validación del cliente, el demonio y la ejecución de la imagen hello-world.

## 3 Instalación y configuración de PostgreSQL

### 3.1 Descarga de la imagen

PostgreSQL se desplegó dentro de un contenedor, por lo que no fue necesario instalar el servidor directamente sobre Fedora. Se eligió la etiqueta postgres:18 en lugar de latest; de esta manera se mantiene fija la versión principal y el procedimiento puede repetirse con resultados previsibles.

```
1 $ docker pull postgres:18
```

La descarga produjo el resumen SHA-256 de la imagen y registró PostgreSQL 18.6 como versión disponible en el momento de la práctica.

### 3.2 Credencial y creación del contenedor

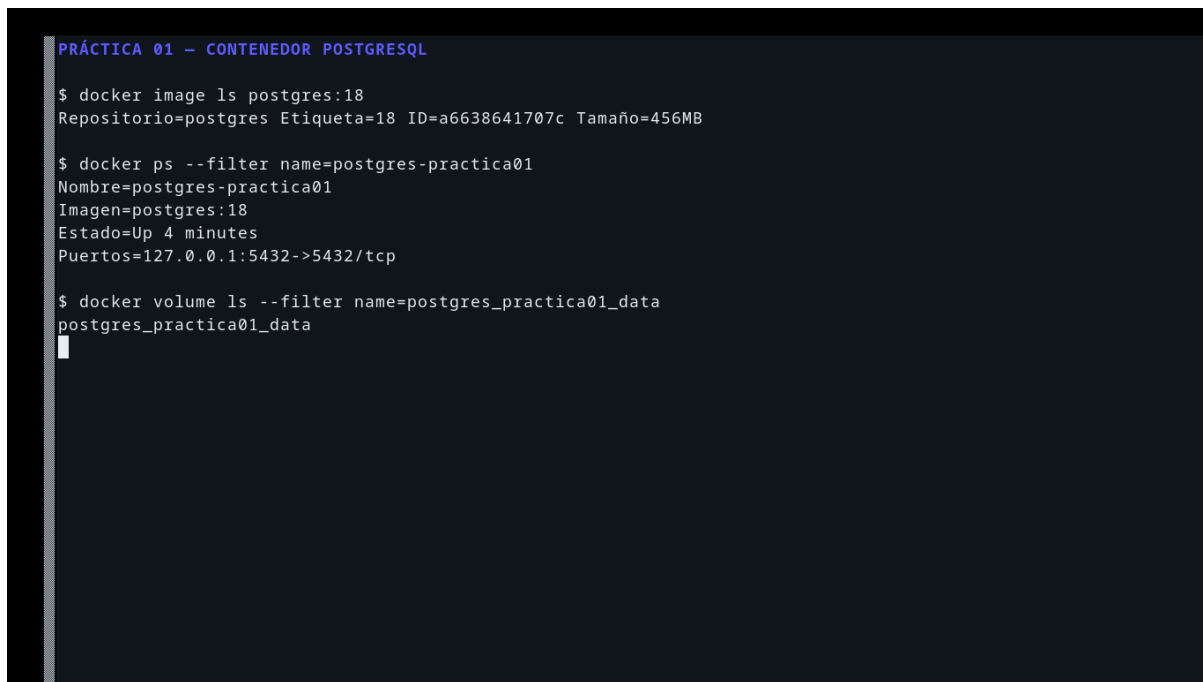
La contraseña se generó aleatoriamente en un archivo temporal. El uso de -env-file impidió que el valor quedara visible en el comando o en la captura de pantalla.

```
1 $ umask 077
2 $ printf 'POSTGRES_PASSWORD=' > /tmp/nipgg_practica01_postgres.env
3 $ openssl rand -base64 24 | tr -d '\n' >> /tmp/nipgg_practica01_postgres.env
4 $ printf '\n' >> /tmp/nipgg_practica01_postgres.env
```

Se creó el contenedor con un nombre descriptivo, un volumen persistente y un mapeo de red restringido a la interfaz local. La restricción 127.0.0.1 evita exponer el servidor directamente a

otros equipos de la red.

```
1 $ docker run -d \  
2   --name postgres-practica01 \  
3   --env-file /tmp/nipgg_practica01_postgres.env \  
4   -p 127.0.0.1:5432:5432 \  
5   -v postgres_practica01_data:/var/lib/postgresql \  
6   postgres:18
```



```
PRÁCTICA 01 - CONTENEDOR POSTGRESQL  
  
$ docker image ls postgres:18  
Repositorio=postgres Etiqueta=18 ID=a6638641707c Tamaño=456MB  
  
$ docker ps --filter name=postgres-practica01  
Nombre=postgres-practica01  
Imagen=postgres:18  
Estado=Up 4 minutes  
Puertos=127.0.0.1:5432->5432/tcp  
  
$ docker volume ls --filter name=postgres_practica01_data  
postgres_practica01_data
```

Figura 3: Imagen, contenedor, puerto local y volumen persistente de PostgreSQL.

### 3.3 Comprobación con psql

Los registros del contenedor indicaron que el sistema estaba listo para aceptar conexiones. A continuación se abrió psql dentro del propio contenedor y se consultaron la base activa, el usuario y la versión del servidor:

```
1 $ docker logs --tail 12 postgres-practica01  
2 $ docker exec postgres-practica01 psql -U postgres -d postgres \  
3   -c 'SELECT current_database(), current_user, version();'
```

```
PRÁCTICA 01 - PRUEBA CON PSQL

$ docker exec postgres-practica01 psql -U postgres -d postgres
current_database | current_user | version
-----+-----+-----
postgres        | postgres     | PostgreSQL 18.6 (Debian 18.6-1.pgdg13+2) on x86_64-pc-linux-gnu, compiled
14.2.0-19) 14.2.0, 64-bit
(1 row)

Resultado: PostgreSQL acepta conexiones correctamente.
```

Figura 4: Consulta de validación ejecutada con psql sobre PostgreSQL 18.6.

## 4 Conexión con DBeaver

### 4.1 Instalación del cliente

Se descargó DBeaver Community 26.2.0 desde el sitio oficial. El archivo RPM se verificó antes de instalarse y, posteriormente, se confirmó el paquete registrado en Fedora.

```
1 $ curl -fL -o /tmp/dbeaver-ce-26.2.0-linux-x86_64.rpm \
2   https://dbeaver.io/files/dbeaver-ce-latest-linux-x86_64.rpm
3 $ rpm -K /tmp/dbeaver-ce-26.2.0-linux-x86_64.rpm
4 $ sudo dnf install /tmp/dbeaver-ce-26.2.0-linux-x86_64.rpm
5 $ rpm -q dbeaver-ce
6 dbeaver-ce-26.2.0-stable.x86_64
```

Al abrir DBeaver por primera vez se descargó el controlador PostgreSQL JDBC 42.7.13. Después se creó una conexión con los parámetros de la Tabla 2.

| Parámetro             | Valor                      |
|-----------------------|----------------------------|
| Nombre de la conexión | PostgreSQL - Práctica 01   |
| Servidor              | localhost                  |
| Puerto                | 5432                       |
| Base de datos         | postgres                   |
| Usuario               | postgres                   |
| Contraseña            | Credencial temporal oculta |

Tabla 2: Parámetros utilizados para la conexión local.

La opción de conexión se ejecutó con la contraseña suministrada desde un archivo de variables temporal. PostgreSQL confirmó tres contextos de DBeaver: la conexión principal, la consulta de metadatos y la consola SQL.

```
PRÁCTICA 01 - DBEAVER Y POSTGRESQL

$ rpm -q dbeaver-ce
dbeaver-ce-26.2.0-stable.x86_64

Conexiones observadas por PostgreSQL:
      application_name          | client_addr | state
-----+-----+-----
DBeaver 26.2.0 - Main <postgres> | 172.17.0.1 | idle
DBeaver 26.2.0 - Metadata <postgres> | 172.17.0.1 | idle
DBeaver 26.2.0 - SQLEditor <PostgreSQL - Pr\xc3\xba1ctica 01> | 172.17.0.1 | idle
(3 rows)

La contraseña no se muestra en esta evidencia.
█
```

Figura 5: Versión de DBeaver y sesiones del cliente observadas por PostgreSQL.

La interfaz mostró la conexión activa en el navegador y abrió una consola asociada al esquema public de la base postgres. Esta evidencia confirma que el controlador JDBC, la autenticación, el puerto y el servidor funcionaron de manera conjunta.



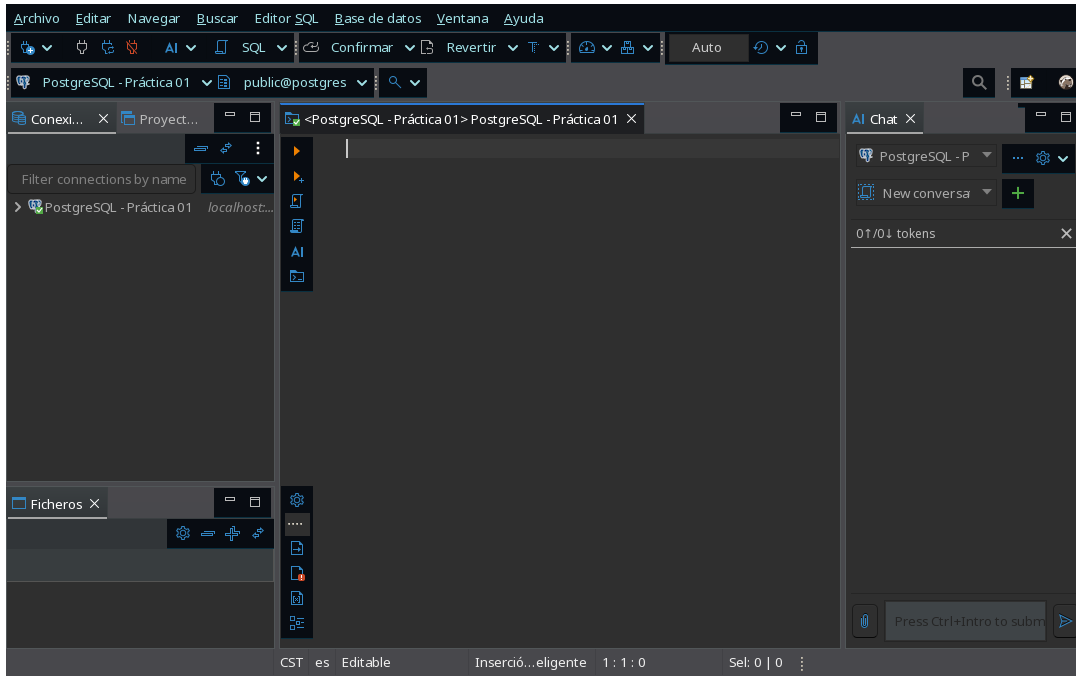


Figura 6: DBeaver Community conectado al servidor PostgreSQL de la práctica.

## 5 Problemas encontrados y comentarios

### 5.1 Incidencias reales

| Incidencia                                      | Causa                                                                                                         | Solución aplicada                                                                                                    |
|-------------------------------------------------|---------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <b>Acceso denegado al socket de Docker</b>      | Aunque el usuario ya pertenecía al grupo docker, la sesión activa todavía no había cargado dicha pertenencia. | Se aplicó el grupo en una subsección con sg docker; un nuevo inicio de sesión produce el mismo efecto.               |
| <b>Instrucciones no específicas para Fedora</b> | La guía proporcionaba ejemplos para Debian, Ubuntu, Windows y macOS.                                          | Se conservaron los objetivos de la práctica y se utilizaron los paquetes y el gestor de servicios propios de Fedora. |

Tabla 3: Problemas observados y soluciones empleadas.

### 5.2 Comentarios finales

El uso de contenedores permitió desplegar PostgreSQL sin incorporar archivos del servidor al sistema operativo anfitrión. El volumen independiente conserva los datos aunque el contenedor se detenga o se sustituya, mientras que el mapeo limitado a localhost reduce la superficie de exposición durante el trabajo local.

La validación se efectuó en tres niveles: ejecución de un contenedor mínimo, consulta directa

con `psql` y conexión mediante DBeaver. Esta secuencia permitió distinguir problemas del motor Docker, del servidor PostgreSQL y del cliente gráfico. Al concluir, la credencial temporal en texto plano se eliminó; DBeaver conservó únicamente su representación dentro del almacén seguro de la aplicación.

**Resultado**

Docker quedó habilitado y activo; PostgreSQL 18.6 permaneció en ejecución bajo el nombre `postgres-practica01`; y DBeaver Community 26.2.0 se conectó correctamente a la base `postgres` mediante JDBC.